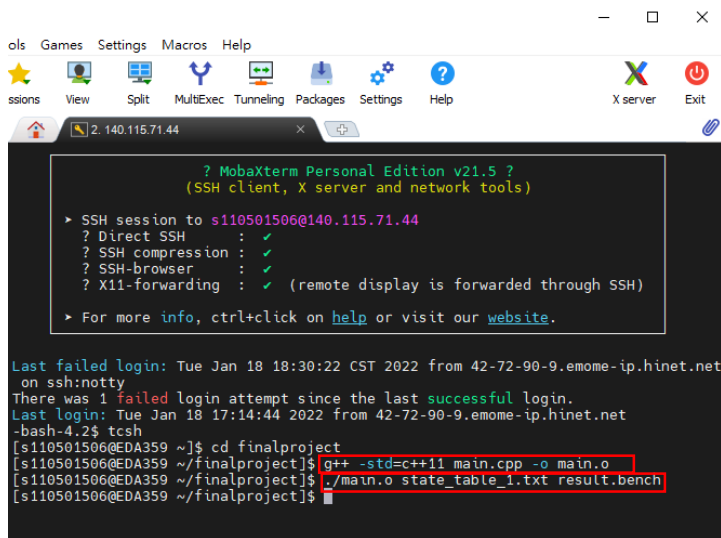


1. How to compile and execute your program.



```
ols Games Settings Macros Help
sessions View Split MultiExec Tunneling Packages Settings Help X server Exit

? MobaXterm Personal Edition v21.5 ?
(SSH client, X server and network tools)

> SSH session to s110501506@140.115.71.44
? Direct SSH : ✓
? SSH compression : ✓
? SSH-browser : ✓
? X11-forwarding : ✓ (remote display is forwarded through SSH)
> For more info, ctrl+click on help or visit our website.

Last failed login: Tue Jan 18 18:30:22 CST 2022 from 42-72-90-9.emome-ip.hinet.net
on ssh:notty
There was 1 failed login attempt since the last successful login.
Last login: Tue Jan 18 17:14:44 2022 from 42-72-90-9.emome-ip.hinet.net
-bash-4.2$ tcsh
[s110501506@EDA359 ~]$ cd finalproject
[s110501506@EDA359 ~/finalproject]$ g++ -std=c++11 main.cpp -o main.o
[s110501506@EDA359 ~/finalproject]$ ./main.o state_table_1.txt result.bench
[s110501506@EDA359 ~/finalproject]$
```

The line in the first block is to compile, and the line in the second block is to execute.

2. The completion of the assignment.

1. Read the state_table file.

```
void table_to_data(char *a){
    ifstream table_in(a,ios::in);
    //store to presentstate
    string ps;
    table_in >> ps>>ps>>ps;
    for(int i=0;i<ps.length();i++){
        if(ps.at(i)==' '){
            psa++;
        }
        for(int i=0;i<psa;i++){
            present[i].name=ps.at(10+i*2);
        }
        if(ps.at(10)=='0'){
            psa=0;
            present[0].name=' ';
        }
        int c=0;
        table_in>> ps;
        while(ps.at(0)!='/'){
            for(int i=0;i<psa;i++){
                present[i].value[c]=stoi(ps);
                present[i].valuamount=c++;
                table_in>>ps;
            }c++;
        }
    }
}
```

↑ The code to store present state. The ways to store other information are similar to these code.

2. Translate state table data to the kmap_in file

```
//nextstate function
if(nextstate[0].valuamount==2){
    for(int i=0;i<nea;i++){
        kmap_in << variablenumber<<endl;
        kmap_in <<nextstate[i].value[0]<<' '<<nextstate[i].value[1]<<endl;
    }
}
else if(nextstate[0].valuamount==4){
    for(int i=0;i<nea;i++){
        kmap_in << variablenumber<<endl;
        kmap_in <<nextstate[i].value[0]<<' '<<nextstate[i].value[1]<<' '<<nextstate[i].value[3]<<' '
        <<nextstate[i].value[2]<<endl;
    }
}
else if(nextstate[0].valuamount==8){
    for(int i=0;i<nea;i++){
        kmap_in << variablenumber<<endl;
        kmap_in <<nextstate[i].value[0]<<' '<<nextstate[i].value[1]<<' '<<nextstate[i].value[3]<<' '
        <<nextstate[i].value[2]<<' '<<nextstate[i].value[4]<<' '<<nextstate[i].value[5]<<' '
        <<nextstate[i].value[7]<<endl;
    }
}
else if(nextstate[0].valuamount==16){
    for(int i=0;i<nea;i++){
        kmap_in << variablenumber<<endl;
        kmap_in <<nextstate[i].value[0]<<' '<<nextstate[i].value[1]<<' '<<nextstate[i].value[3]<<' '
        <<nextstate[i].value[2]<<' '<<nextstate[i].value[4]<<' '<<nextstate[i].value[5]<<' '
        <<nextstate[i].value[7]<<' '<<nextstate[i].value[6]<<' '<<nextstate[i].value[12]<<' '
        <<nextstate[i].value[13]<<' '<<nextstate[i].value[15]<<' '<<nextstate[i].value[14]<<' '
        <<nextstate[i].value[8]<<' '<<nextstate[i].value[9]<<' '<<nextstate[i].value[11]<<' '
        <<nextstate[i].value[10]<<endl;
    }
}
```

↑ Dealing with nextstate function.(the codes to deal with output function are

similar to these)

3. Read the Kmap_out file.

```
void store_kmapout(){
    ifstream kmap_out("kmap_out.txt", ios::in);
    string namearr[100];
    char digitarr[100];
    for(int i=0; i<psa; i++){
        namearr[i]=present[i].name;
    }
    for(int i=psa; i<cin+psa; i++){
        namearr[i]=input[i-psa].name;
    }

    //store N
    int c=0, m=1, fn=0;
    while(kmap_out>>digitarr[0]){
        if(digitarr[0]!='F'){
            functionamount++;
            c=0;
            fn=0;
        }
        else{
            fn++;
        }
        for(int i=0; i<psa+ina; i++){
            kmap_out>>digitarr[i];
        }
        for(int i=0; i<psa+ina; i++){
            if(functionamount!=name-"N" to_string(m);
            if(digitarr[i]!='0'){
                inveter[inveteramount].name="I"+namearr[i];
                inveteramount++;
                N[andgateamount].content[i]="I"+namearr[i];
            }
            else if(digitarr[i]=='1'){
                N[andgateamount].content[i]=namearr[i];
            }
            andgateamount++; c++; m++;
            function_[functionamount-1].content[c-1]=N[andgateamount-1].name;
            if(fn==1)
                orgateamount++;
        }
    }
}
```

↑ The function to store the output and nextstate data.

```
void change_functionname_and_Nname(){
    int i=0;
    string buf;
    Nname=andgateamount;
    for(int i=0; i<andgateamount; i++){
        for(int x=0; x<100; x++){
            if(N[i].content[i]!="0"){
                buf=N[i].content[i];
                x2=i;
            }
            if(x==1){
                andgateamount--;
                for(int k1=0; k1<functionamount; k1++){
                    for(int k2=0; k2<10; k2++){
                        if(function_[k1].content[k2]==N[i2].name)
                            function_[k1].content[k2]=buf;
                    }
                }
                N[i].name=buf;
            }
        }
    }

    //Delete with case N
    int c=0;
    for(int i=0; i<andgateamount; i++){
        for(int x=0; x<10; x++){
            if(N[i].content[0]==N[j].content[0] && N[i].content[1]==N[j].content[1] && N[i].content[2]==N[j].content[2] && N[i].content[3]==N[j].content[3]){
                for(int k1=0; k1<functionamount; k1++){
                    for(int k2=0; k2<10; k2++){
                        if(function_[k1].content[k2]==N[i].name)
                            function_[k1].content[k2]=N[j].name;
                    }
                }
                save_N(c-hj);
                c++;
                N[i]=N[j];
                andgateamount--;
            }
        }
    }

    //Delete the same functionname
    for(int i=0; i<inveteramount; i++){
        for(int j=i+1; j<inveteramount; j++){
            if(inveter[j].name==inveter[i].name){
                inveter[j].name="";
            }
        }
    }
}
```

↑ The function to check if there are same andgate.

4. Output the circuit to the bench file.

5. Connect main and the kmap.cpp file.

3. The hardness of this assignment and how you overcome it.

1. I cannot download the Workstation program successfully. Therefore, I had to ask my friends to test my code. Fortunately, I have a friendly roommate to do me this favor.
2. There seems to be some difference between Cold::Blocks and Workstation when compiling. For example, I can compare the character variable with NULL in Cold::Blocks, but it doesn't work in Workstation. I initialized the character variable name to space(' ') and change the following code a little bit. By doing so, the code work in both Cold::Blocks and Workstation.
3. The most difficult part for me is to determine the form for storing the information. Before starting writing this assignment, I take a long time to figure out this problem. I checked the introduction carefully to ensure that I do not miss any information needed to

4. Any suggestions about this programming assignment?

I think there can be more testcases for us to test our codes. Otherwise, it's not a simple thing to make a complex state_table file and check if the answer is correct or not.